

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México



Momento de Retroalimentación: Análisis y Reporte sobre el desempeño del modelo

Diagnóstico de sesgo, varianza y ajuste de un Random Forest, y su mejora mediante regularización

Alumno:

Juan Pablo Pérez Gutiérrez – A01800483

Curso: Inteligencia artificial avanzada para la ciencia de datos

Profesor: Jorge Adolfo Ramírez Uresti

Grupo: 601

16 de Septiembre del 2026

1. Objetivo y modelo analizado

El objetivo de esta entrega es analizar a fondo el desempeño de uno de los modelos implementados en el módulo: diagnosticar su grado de sesgo, su grado de varianza y su nivel de ajuste, y a partir de ese diagnóstico aplicar técnicas de regularización y ajuste de hiperparámetros para mejorarlo, documentando con indicadores y gráficas qué cambió.

Se eligió el **Random Forest de la implementación con scikit-learn** (`main_framework.py`) sobre el dataset Spaceship Titanic, por tres razones. Primero, Random Forest es el algoritmo central de las dos entregas anteriores. Segundo, con sus hiperparámetros por defecto es un caso de estudio ideal: crece cada árbol hasta el fondo, así que presenta un sobreajuste marcado que se puede diagnosticar y corregir con claridad. Tercero, el framework proporciona las herramientas necesarias para construir curvas de aprendizaje y de validación con validación cruzada de forma rigurosa.

El problema es de clasificación binaria: predecir si un pasajero fue transportado a otra dimensión (Transported) a partir de 13 características como su planeta de origen, si viajaba en criosueño, su cabina, su edad y su gasto en las amenidades de la nave. Todo el análisis se ejecuta con un solo comando (`python analisis_desempeno.py`) y es reproducible con semilla fija.

2. Separación Train / Validation / Test

El archivo `data/train.csv` (8,693 pasajeros) se dividió en tres subconjuntos de forma estratificada, es decir, conservando en cada uno la misma proporción de pasajeros transportados. El archivo `data/test.csv` de Kaggle no se usa para evaluar porque no contiene la variable objetivo.

Subconjunto	Muestras	% del total	% Transported	Función en el análisis
Entrenamiento	6,085	70 %	50.37 %	Ajustar los modelos; validación cruzada de 5 pliegues para curvas y búsqueda
Validación	1,304	15 %	50.31 %	Medir la brecha contra entrenamiento y la descomposición sesgo-varianza
Prueba	1,304	15 %	50.38 %	Evaluación final, una sola vez por modelo

Cada subconjunto tiene un papel distinto y ninguno se usa para dos cosas. El de entrenamiento es el único que ven los modelos al aprender; dentro de él se usa además validación cruzada estratificada de 5 pliegues (siempre los mismos pliegues para todos los modelos) para construir las curvas y elegir hiperparámetros sin tocar los otros dos conjuntos. El de validación sirve para diagnosticar: comparar su exactitud contra la de entrenamiento es lo que revela la varianza. El de prueba queda reservado para confirmar al final que las conclusiones se sostienen sobre datos que no participaron en ninguna decisión.

3. Cómo se diagnostican el sesgo, la varianza y el ajuste

El error de generalización de un modelo se puede pensar como la suma de tres componentes: el sesgo (error sistemático por usar un modelo demasiado simple para el patrón real), la varianza (sensibilidad del modelo a los datos particulares con que se entrenó) y el ruido irreducible del problema. Para diagnosticar cada uno se usaron varios indicadores que se complementan, en lugar de uno solo.

Indicador	Qué mide	Cómo se interpreta
Error de entrenamiento (1 – exactitud en train)	Sesgo	Si el modelo no logra ajustar ni siquiera los datos con que aprendió, es demasiado simple
Brecha entrenamiento – validación	Varianza	Una brecha grande indica que el modelo aprendió particularidades del entrenamiento que no se repiten en datos nuevos
Curva de aprendizaje	Sesgo y varianza	Curvas separadas: varianza alta. Curvas juntas y bajas: sesgo alto. Curvas juntas y altas: buen ajuste
Curvas de validación	Ajuste según complejidad	Muestran la transición underfit → fit → overfit al variar cada hiperparámetro
Descomposición sesgo-varianza por bootstrap	Sesgo y varianza directamente	20 modelos entrenados con remuestreos distintos; mide cuánto se equivocan en promedio y cuánto cambian sus predicciones
Dispersión entre pliegues de validación cruzada	Estabilidad	Qué tanto varía la exactitud al cambiar la partición

3.1 Umbrales para clasificar en bajo, medio y alto

Nivel	Sesgo (error de entrenamiento)	Varianza (brecha train – val)
Bajo	< 0.10	< 0.03
Medio	$0.10 - 0.20$	$0.03 - 0.08$
Alto	≥ 0.20	≥ 0.08

Los umbrales no son arbitrarios. En la entrega anterior se entrenaron tres familias de modelos muy distintas (regresión logística, Random Forest y Gradient Boosting) y las tres se estancaron alrededor de 80 % de exactitud. Que modelos con capacidades tan diferentes choquen contra el mismo techo indica que una parte cercana al 20 % del error es irreducible con estas características: hay pasajeros con perfiles prácticamente idénticos y desenlaces opuestos. Por eso el sesgo se considera alto solo si el error de entrenamiento alcanza ese 20 %, es decir, si el modelo no logra aprender ni lo que es aprendible. Para la varianza, una brecha menor a 3 puntos está dentro de lo que se espera por simple variación de muestreo, mientras que una mayor a 8 puntos es un síntoma claro de memorización.

El nivel de ajuste se deduce de la combinación: varianza alta implica overfit; sesgo alto con varianza baja implica underfit; cualquier otra combinación corresponde a un modelo razonablemente ajustado (fit).

4. Diagnóstico del modelo base

El modelo base es un Random Forest con los hiperparámetros por defecto de scikit-learn: 100 árboles, sin límite de profundidad (`max_depth=None`), una muestra mínima por hoja (`min_samples_leaf=1`) y $\sqrt{n}$ características por nodo. Con esa configuración cada árbol crece hasta aislar prácticamente cada pasajero: la profundidad promedio de sus árboles es de 27.7 niveles y tienen en promedio 1,106 hojas cada uno, para un conjunto de entrenamiento de 6,085 filas.

Conjunto	Exactitud	Precisión	Sensibilidad	F1	AUC
Entrenamiento	0.9998	1.0000	0.9997	0.9998	1.0000
Validación	0.8037	0.8344	0.7607	0.7959	0.8925
Prueba	0.7876	0.8199	0.7412	0.7786	0.8769

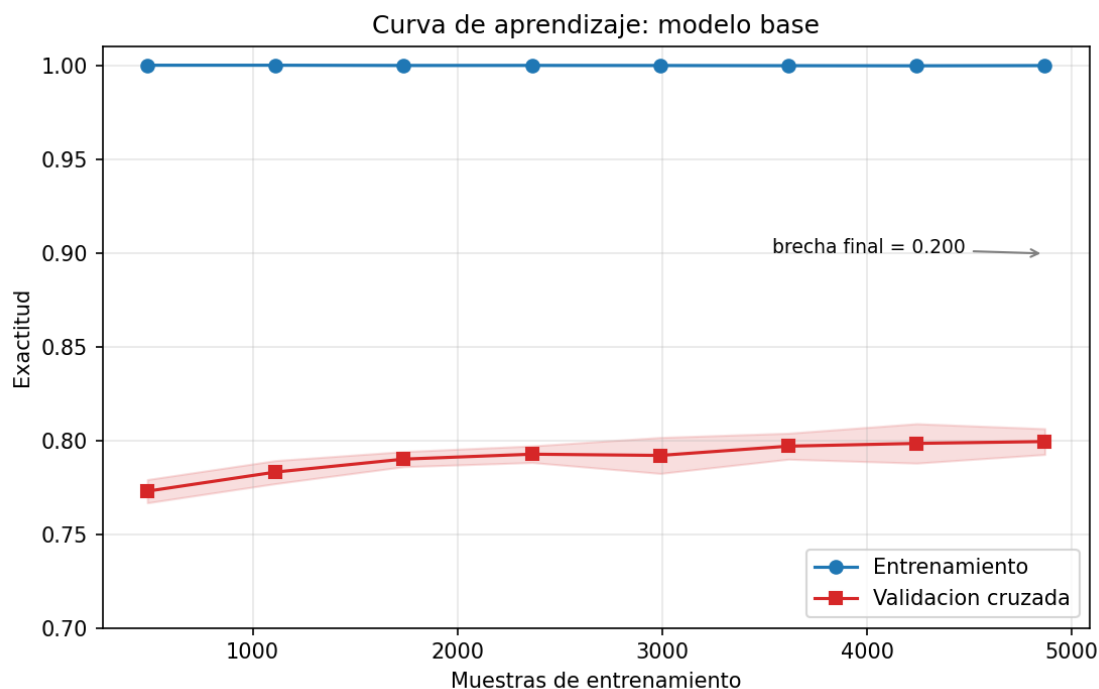


Figura 1. Curva de aprendizaje del modelo base (media $\pm$ desviación estándar sobre 5 pliegues).

Sesgo: BAJO. El error de entrenamiento es de 0.0002: el modelo clasifica correctamente 99.98 % de los pasajeros con que aprendió, con un AUC de 1.0000. Un modelo con sesgo alto no podría ajustar sus propios datos; este los ajusta a la perfección. Su capacidad para representar patrones complejos no es la limitante.

Varianza: ALTA. La brecha entre entrenamiento (0.9998) y validación (0.8037) es de 0.1962, casi 20 puntos porcentuales, más del doble del umbral de varianza alta. La curva de aprendizaje lo muestra de forma inequívoca: la curva de entrenamiento está clavada en 1.0 para cualquier tamaño de muestra, mientras que la de validación se queda alrededor de 0.79. Las dos curvas nunca se acercan. La descomposición por bootstrap lo confirma: en promedio 6.6 % de las predicciones de validación cambian simplemente por entrenar con otro remuestreo de los mismos datos.

Nivel de ajuste: OVERFIT. Sesgo bajo con varianza alta es la firma del sobreajuste. El modelo no aprendió solo el patrón general del problema, sino también el ruido particular de cada pasajero del entrenamiento: un árbol con más de mil hojas para seis mil filas prácticamente tiene una regla dedicada a cada grupo pequeño de pasajeros. Esas reglas no se repiten en datos nuevos, y de ahí la caída de casi 20 puntos.

Un detalle de la curva de aprendizaje también es informativo: la curva de validación sigue subiendo ligeramente al final (de 0.773 con 486 muestras a 0.799 con 4,868). Más datos ayudarían un poco, que es lo esperable en un modelo con varianza alta, pero conseguir más datos no es una opción; la alternativa es reducir la complejidad del modelo, que es lo que hace la regularización.

5. Curvas de validación: underfit, fit y overfit

Antes de buscar la mejor configuración conviene entender qué hace cada hiperparámetro de regularización. Una curva de validación entrena el modelo variando un solo hiperparámetro y compara la exactitud de entrenamiento con la de validación cruzada. Las tres curvas siguientes muestran, cada una, el recorrido completo entre los tres niveles de ajuste.

5.1 Profundidad máxima (max_depth)

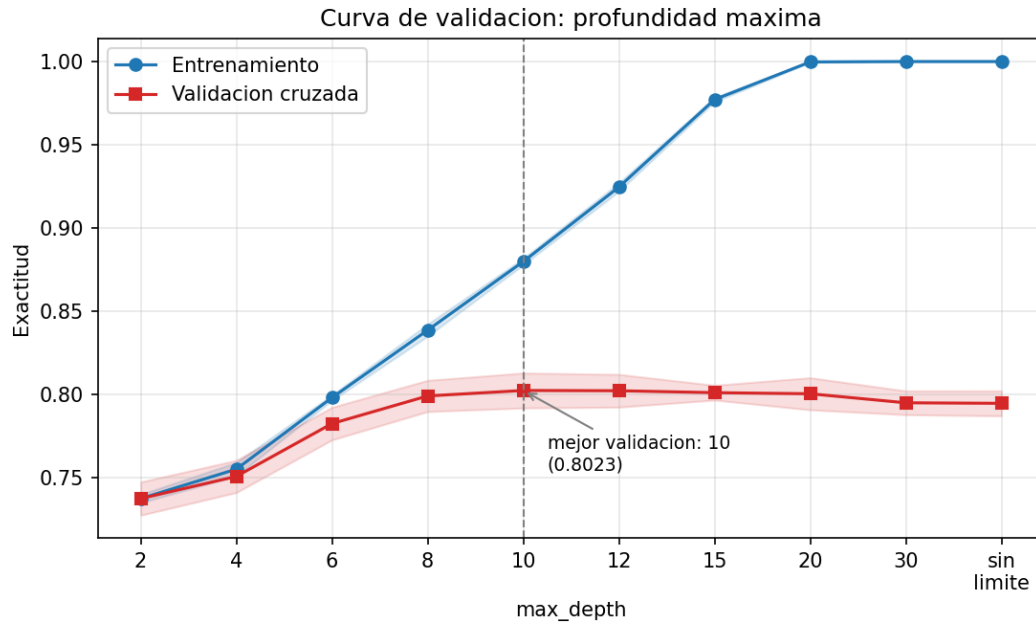


Figura 2. Curva de validación para la profundidad máxima de los árboles.

max_depth	Entrenamiento	Validación CV	Brecha	Zona
2	0.7374	0.7374	0.0000	Underfit
4	0.7551	0.7507	0.0044	Underfit
6	0.7980	0.7824	0.0156	Transición
8	0.8385	0.7990	0.0395	Fit
10	0.8797	0.8023	0.0774	Fit
12	0.9244	0.8021	0.1223	Overfit
15	0.9770	0.8010	0.1760	Overfit
20	0.9996	0.8003	0.1993	Overfit
30	0.9998	0.7949	0.2049	Overfit
sin limite	0.9998	0.7946	0.2053	Overfit

Esta es la gráfica que mejor ilustra los tres regímenes. Con profundidad 2 o 4 los árboles son tan simples que ambas curvas coinciden (brecha prácticamente cero) pero son bajas, alrededor de 0.74–0.75: el modelo no alcanza a capturar el patrón. Es un underfit, con sesgo alto y varianza baja. Entre 8 y 10 niveles la validación alcanza su máximo (0.8023) con una brecha moderada: es la zona de buen ajuste. A partir de 12 niveles la exactitud de entrenamiento sigue subiendo hasta 1.0 pero la de validación ya no mejora y termina bajando: todo lo que se agrega es memorización, el overfit del modelo base (que corresponde al extremo derecho, sin límite).

5.2 Mínimo de muestras por hoja (min_samples_leaf)

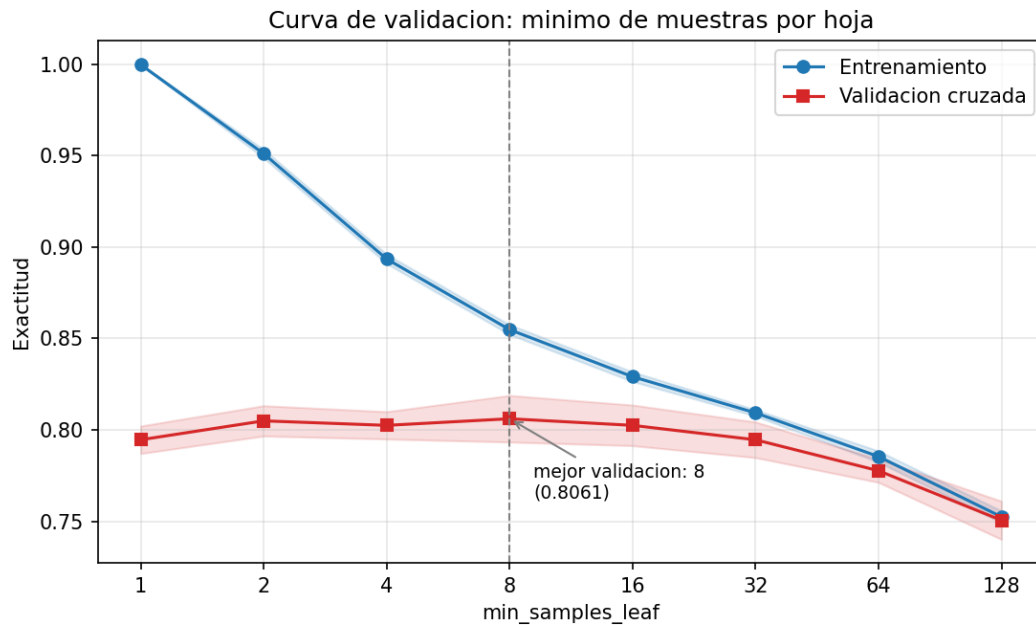


Figura 3. Curva de validación para el mínimo de muestras por hoja.

Aquí el eje de complejidad va al revés: un valor pequeño permite hojas con un solo pasajero (máxima complejidad) y uno grande obliga a promediar muchos. Con 1 muestra por hoja la brecha es de 0.205 (overfit); con 8 la validación llega a su máximo (0.8061) y la brecha baja a 0.049; con 128 las hojas son tan grandes que ambas curvas caen juntas a 0.751 (underfit). Es el hiperparámetro más efectivo individualmente, porque ataca directamente la causa del sobreajuste: que existan hojas que describen a un solo pasajero.

5.3 Poda por costo-complejidad (ccp_alpha)

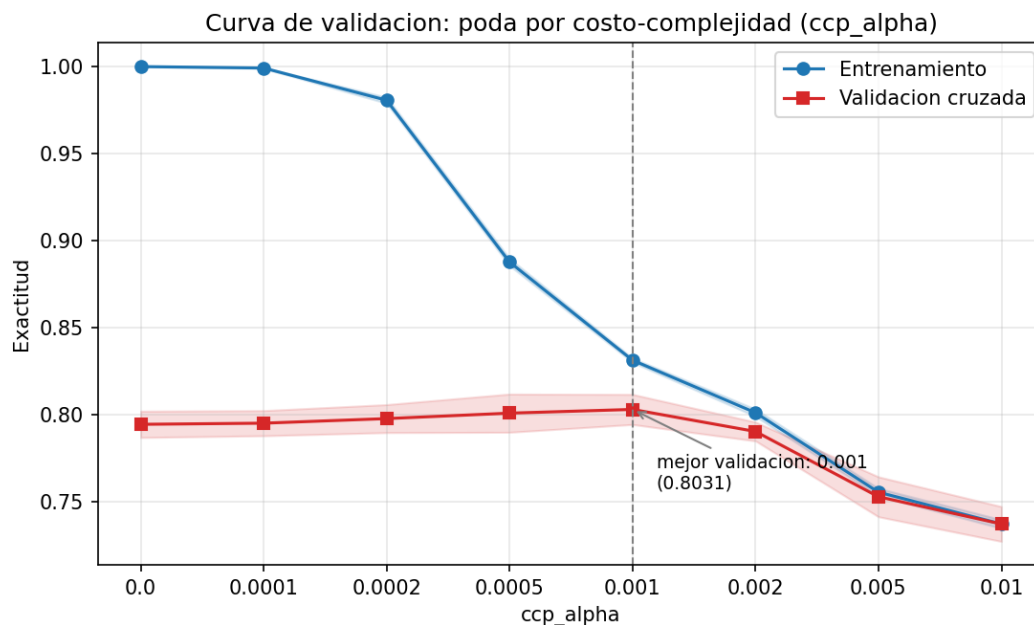


Figura 4. Curva de validación para el parámetro de poda por costo-complejidad.

ccp_alpha es la técnica de regularización en el sentido más estricto: agrega una penalización proporcional al número de hojas del árbol, de modo que una rama solo se conserva si la mejora en impureza que aporta supera ese costo. Es el equivalente, para árboles, de la penalización L1 o L2 en un modelo lineal. Con $\alpha = 0$ no hay poda (overfit, brecha 0.205); con $\alpha = 0.001$ la validación alcanza 0.8031 con brecha de apenas 0.028; con $\alpha = 0.01$ la poda elimina tanto que el árbol queda reducido a unas pocas reglas y ambas curvas caen a 0.737 (underfit, idéntico al de profundidad 2).

Las tres curvas cuentan la misma historia: existe una zona intermedia de complejidad donde la validación es máxima, y los hiperparámetros por defecto están fuera de ella, en el extremo del sobreajuste. También muestran algo importante para la siguiente sección: la mejora máxima en validación que se puede obtener regularizando es de alrededor de un punto porcentual, mientras que la reducción posible de la brecha es de más de quince.

6. Regularización y ajuste de hiperparámetros

Las curvas de validación estudian cada hiperparámetro por separado, pero estos interactúan: si se limita la profundidad, el mínimo por hoja óptimo cambia. Por eso la configuración final se eligió con una búsqueda conjunta, RandomizedSearchCV, sobre siete hiperparámetros, con validación cruzada estratificada de 5 pliegues dentro del conjunto de entrenamiento.

Hiperparámetro	Valores explorados	Qué regulariza
n_estimators	200, 300, 500	Más árboles promedian más y reducen varianza

max_depth	8, 10, 12, 15, 20, sin límite	Profundidad de cada árbol
min_samples_leaf	1, 2, 4, 8, 16	Tamaño mínimo de cada regla final
min_samples_split	2, 5, 10, 20	Muestras necesarias para dividir un nodo
max_features	sqrt, 0.3, 0.5	Decorrelación entre árboles
max_samples	100 %, 50 %, 70 %	Tamaño de cada muestra bootstrap
ccp_alpha	0, 0.0001, 0.0002, 0.0005	Poda por costo-complejidad

El espacio completo tiene 12,960 combinaciones. Una búsqueda exhaustiva habría requerido 64,800 ajustes, así que se muestrearon 60 combinaciones al azar (300 ajustes, 2.7 minutos). La búsqueda aleatoria es preferible a la de rejilla cuando hay muchos hiperparámetros, porque explora más valores distintos de cada uno con el mismo presupuesto.

Las cinco mejores configuraciones encontradas fueron:

max_depth	min_leaf	min_split	max_feat.	max_samp.	ccp_alpha	Val. CV	Brecha
sin límite	4	20	sqrt	50 %	0.0002	0.8058	0.0368
sin límite	4	20	sqrt	100 %	0	0.8058	0.0632
8	2	5	0.3	70 %	0.0001	0.8053	0.0393
10	2	10	0.3	70 %	0	0.8048	0.0605
20	4	20	0.3	70 %	0.0001	0.8048	0.0597

Las dos primeras configuraciones empataron en exactitud de validación cruzada, pero la ganadora tiene una brecha de 0.037 contra 0.063 de la segunda. La diferencia está en que usa max_samples = 50 % y una poda ccp_alpha = 0.0002: con el mismo desempeño, es la opción más regularizada, que es justo lo que busca este análisis.

Configuración final del modelo regularizado:

- **n_estimators = 300.** Tres veces más árboles que el modelo base; más votos promediados reducen la varianza del ensamble sin aumentar el sesgo.
- **min_samples_leaf = 4 y min_samples_split = 20.** Ninguna regla final puede describir a menos de 4 pasajeros, y un nodo con menos de 20 no se divide. Es lo que elimina las hojas de un solo pasajero.
- **max_samples = 0.5.** Cada árbol se entrena con solo la mitad de las filas. Esto hace a los árboles individualmente más simples y, sobre todo, más distintos entre sí, lo que potencia el efecto del promedio.
- **ccp_alpha = 0.0002.** Poda ligera por costo-complejidad que elimina ramas cuya contribución no justifica la complejidad que añaden.
- **max_depth = sin límite y max_features = sqrt.** La profundidad no hizo falta limitarla directamente: las otras restricciones ya frenan el crecimiento (profundidad promedio 16.5 contra 27.7).

El efecto sobre la estructura del modelo es drástico: el número promedio de hojas por árbol pasó de 1,106 a 128, una reducción de 88 %.

7. Diagnóstico del modelo regularizado

Conjunto	Exactitud	Precisión	Sensibilidad	F1	AUC
Entrenamiento	0.8459	0.8527	0.8388	0.8457	0.9318
Validación	0.8021	0.8179	0.7805	0.7988	0.8979
Prueba	0.7860	0.7872	0.7884	0.7878	0.8875

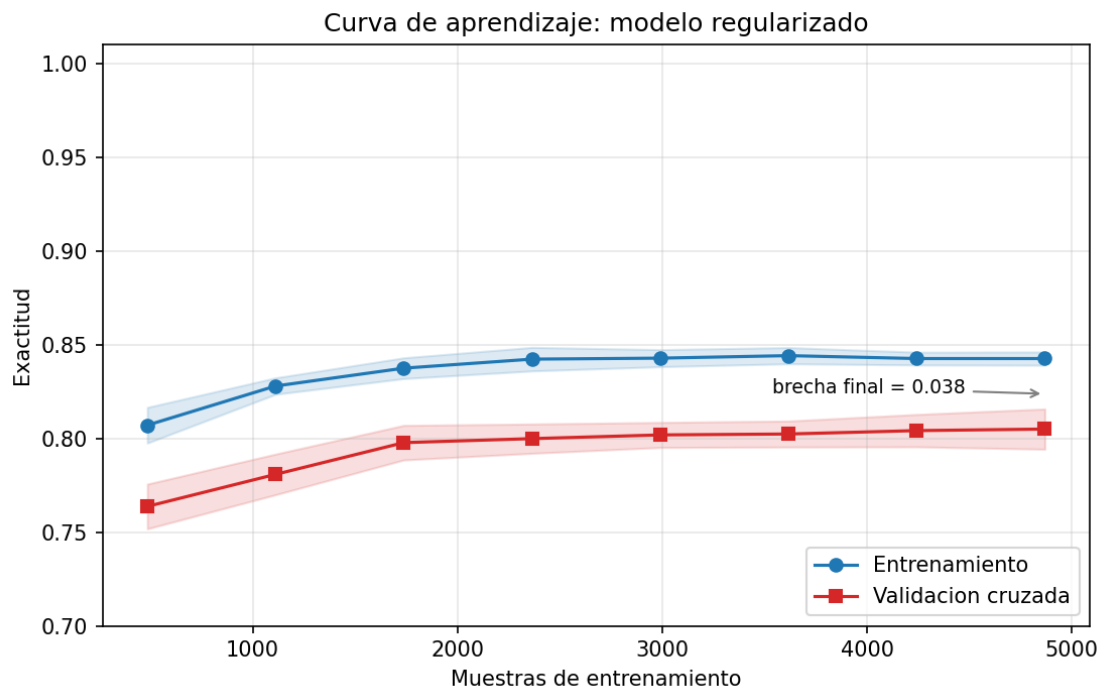


Figura 5. Curva de aprendizaje del modelo regularizado.

Sesgo: MEDIO. El error de entrenamiento subió de 0.0002 a 0.1541. Este aumento es intencional: la regularización consiste precisamente en impedir que el modelo se ajuste a todos los detalles del entrenamiento. Sigue estando por debajo del techo de 20 % del problema, así que el modelo no está desaprovechando patrones aprendibles; la curva de aprendizaje muestra que la exactitud de entrenamiento se estabiliza en 0.843 en lugar de crecer. La descomposición por bootstrap indica que el componente sistemático del error apenas cambió ($0.1925 \rightarrow 0.1994$).

Varianza: MEDIA. La brecha entre entrenamiento y validación bajó de 0.1962 a 0.0437, una reducción de 78 %. En la curva de aprendizaje las dos líneas ahora convergen a medida que crece el entrenamiento, que es la señal de un modelo que generaliza. La varianza medida por bootstrap bajó de 0.0662 a 0.0365 (45 % menos): las predicciones del modelo dependen mucho menos de qué filas concretas le tocaron al entrenar. Queda en nivel medio y no bajo porque persiste una brecha de 4 puntos, inevitable en un ensamble de árboles con esta capacidad.

Nivel de ajuste: FIT. Sesgo medio y varianza media, con las dos curvas de aprendizaje convergiendo hacia el techo del problema y la validación en su máximo, corresponden a un

modelo bien ajustado: suficientemente complejo para capturar el patrón y suficientemente restringido para no memorizar el ruido.

8. Comparación antes y después de regularizar

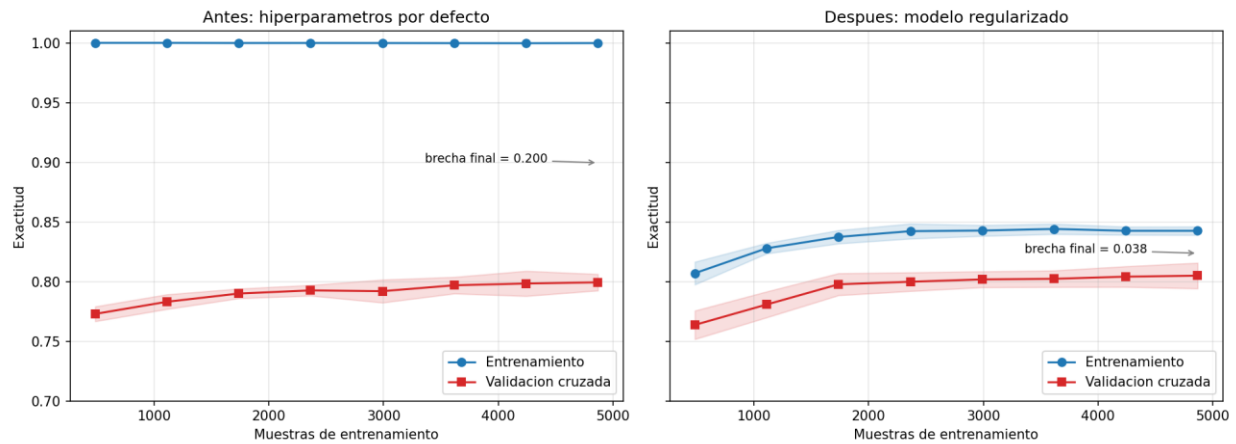


Figura 6. Curvas de aprendizaje antes (izquierda) y después (derecha) de regularizar, en la misma escala.

Vistas en la misma escala, las curvas de aprendizaje resumen todo el análisis. A la izquierda, dos curvas paralelas separadas por 20 puntos que nunca se acercan: overfit. A la derecha, la curva de entrenamiento baja del techo artificial de 1.0 y la de validación sube ligeramente, de modo que la brecha se reduce a menos de 4 puntos.

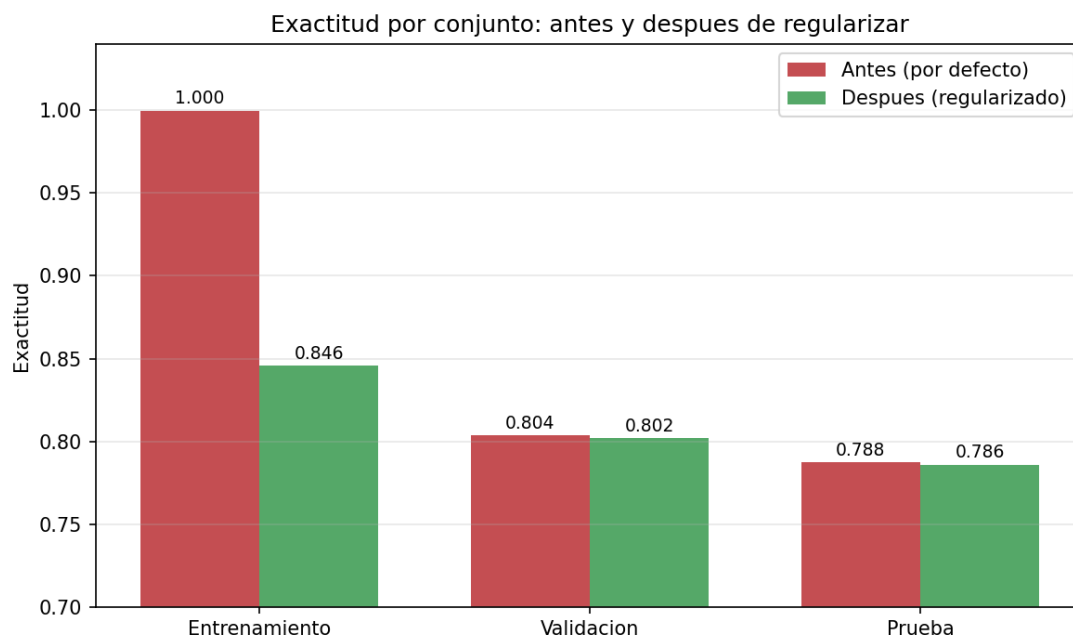


Figura 7. Exactitud en entrenamiento, validación y prueba, antes y después.

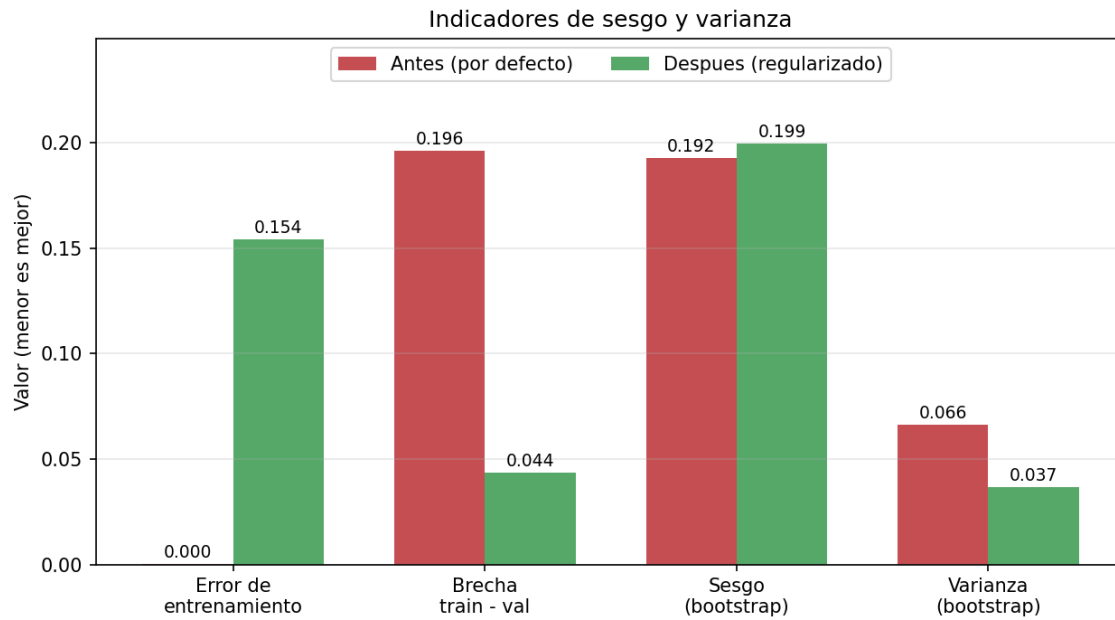


Figura 8. Indicadores de sesgo y varianza antes y después (menor es mejor).

Indicador	Antes	Después	Cambio
Exactitud entrenamiento	0.9998	0.8459	-0.1540
Exactitud validación	0.8037	0.8021	-0.0015
Exactitud prueba	0.7876	0.7860	-0.0015
Exactitud validación cruzada (media)	0.7946	0.8058	+0.0112
AUC prueba	0.8769	0.8875	+0.0106
F1 prueba	0.7786	0.7878	+0.0093
Sensibilidad prueba	0.7412	0.7884	+0.0472
Brecha train – validación	0.1962	0.0437	-0.1525
Varianza (bootstrap)	0.0662	0.0365	-0.0297
Sesgo (bootstrap)	0.1925	0.1994	+0.0069
Hojas promedio por árbol	1,106	128	-978
Diagnóstico	sesgo bajo, varianza alta, overfit	sesgo medio, varianza media, fit	

8.1 Estabilidad entre pliegues

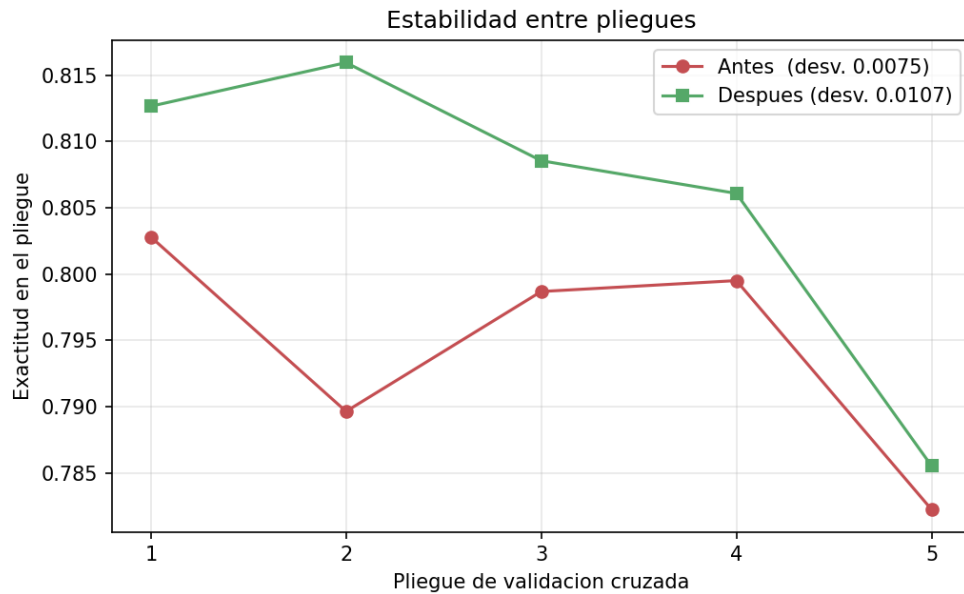


Figura 9. Exactitud de cada pliegue de validación cruzada. Los pliegues son los mismos para ambos modelos.

Pliegue	Antes	Después	Diferencia
1	0.8028	0.8127	+0.0099
2	0.7896	0.8159	+0.0263
3	0.7987	0.8085	+0.0099
4	0.7995	0.8061	+0.0066
5	0.7823	0.7855	+0.0033

Como los dos modelos se evaluaron sobre exactamente los mismos cinco pliegues, la comparación es pareada: cada pliegue es un experimento independiente donde los dos modelos enfrentan los mismos datos. El modelo regularizado ganó en 5 de los 5 pliegues, con una mejora media de 1.1 puntos. Es la evidencia más sólida de mejora del análisis, porque se basa en las 6,085 filas de entrenamiento y no depende de una sola partición.

La desviación estándar entre pliegues subió ligeramente ($0.0075 \rightarrow 0.0107$), pero no por inestabilidad: la mejora fue desigual, de 2.6 puntos en el pliegue 2 y de solo 0.3 en el pliegue 5, lo que abre la dispersión aun cuando todos los pliegues mejoraron. Con solo cinco valores, este indicador es mucho menos confiable que la descomposición por bootstrap, que sí muestra una reducción clara de la varianza.

8.2 Matrices de confusión en prueba

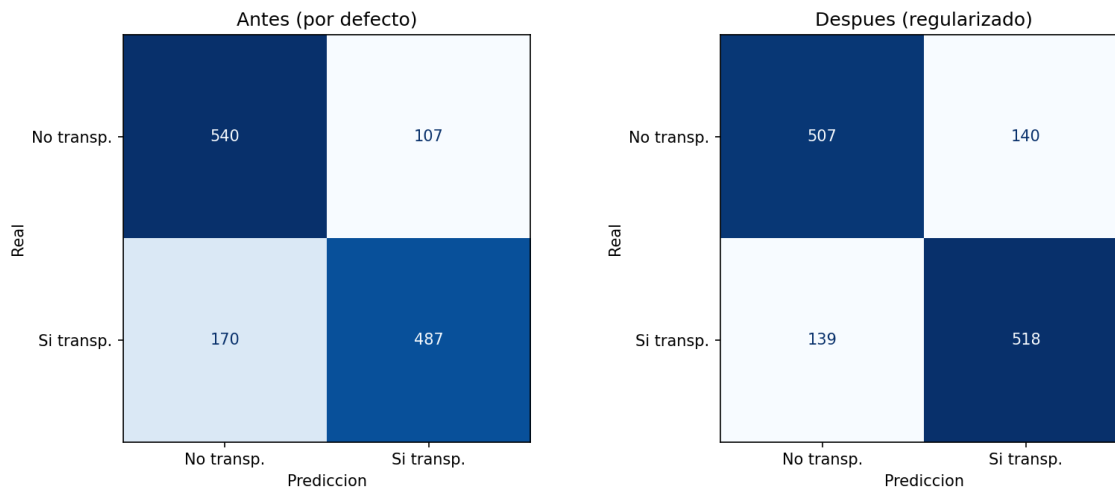


Figura 10. Matrices de confusión sobre el conjunto de prueba.

El cambio en las matrices revela un efecto de la regularización que la exactitud sola esconde. El modelo base cometía 170 falsos negativos y 107 falsos positivos: tenía un sesgo claro hacia predecir «no transportado». El modelo regularizado comete 139 y 140: sus errores están prácticamente equilibrados entre las dos clases. La sensibilidad subió de 0.7412 a 0.7884, casi cinco puntos. La explicación es que los árboles sobreajustados producen probabilidades extremas y poco calibradas, que se reparten mal alrededor del umbral de 0.5; al exigir hojas con varios pasajeros, las probabilidades se suavizan y el umbral separa mejor las dos clases, lo que también explica la mejora del AUC (0.8769 $\rightarrow$ 0.8875).

9. Análisis: qué mejoró y qué no

La regularización mejoró de forma clara:

- La generalización: brecha entre entrenamiento y validación de 0.196 a 0.044 (−78 %).
- La varianza: de 0.0662 a 0.0365 en la descomposición por bootstrap (−45 %), a cambio de un aumento del sesgo de solo 0.0069.
- La exactitud en validación cruzada: 0.7946 $\rightarrow$ 0.8058, con mejora en 5 de 5 pliegues.
- La calidad del ordenamiento y el balance de errores en prueba: AUC +0.0106, F1 +0.0093, sensibilidad +0.0472.
- La simplicidad del modelo: 88 % menos hojas por árbol para el mismo nivel de desempeño.

Lo que no cambió:

La exactitud en los conjuntos de validación y prueba se mantuvo prácticamente igual (−0.0015 y −0.0015). Sobre 1,304 pasajeros, una diferencia de 0.0015 equivale a 2 pasajeros, y el error estándar de una exactitud cercana a 0.79 medida sobre ese tamaño de muestra es de ± 0.011 . La diferencia es unas siete veces menor que el error de medición: no es distinguible de cero. Las

métricas con más poder estadístico (validación cruzada sobre 6,085 filas con comparación pareada, AUC que usa todas las probabilidades) sí muestran mejora.

Este resultado tiene una lectura importante y es coherente con las curvas de validación de la sección 5: en este problema la regularización no puede aumentar mucho la exactitud porque el modelo base ya estaba cerca del techo de ~80 % impuesto por el ruido irreducible. Random Forest es de por sí resistente al sobreajuste, porque el promedio de muchos árboles compensa buena parte de la varianza de cada uno. Lo que la regularización logró fue llegar a ese mismo techo de forma sana: con un modelo que ya no memoriza, cuyas predicciones son estables ante cambios en los datos y cuyos errores están equilibrados. Esa es la diferencia entre un modelo que funciona en este conjunto de prueba y uno en el que se puede confiar con datos nuevos.

10. Conclusiones

- **Modelo base: sesgo bajo, varianza alta, overfit.** Árboles de 28 niveles y 1,106 hojas que memorizan el entrenamiento (0.9998) y caen casi 20 puntos en validación (0.8037).
- **Modelo regularizado: sesgo medio, varianza media, fit.** Entrenamiento 0.8459 y validación 0.8021, con curvas de aprendizaje que convergen.
- **Las curvas de validación mostraron los tres regímenes.** Profundidad 2–4, ccp_alpha 0.01 o 128 muestras por hoja producen underfit; la zona intermedia produce buen ajuste; los valores por defecto producen overfit.
- **La regularización redujo la brecha 78 % y la varianza 45 %,** mejoró la validación cruzada en los 5 pliegues, el AUC, el F1 y la sensibilidad, y equilibró los errores entre clases, manteniendo la exactitud en prueba dentro del margen de error.
- **El techo del problema está en los datos, no en el modelo.** Para superar el ~80 % haría falta información nueva, por ejemplo aprovechar que los pasajeros del mismo grupo de viaje tienden a compartir deslance, más que seguir ajustando hiperparámetros.

Anexo. Reproducción del análisis

El análisis completo está en `analisis_desempeno.py` y se ejecuta con un intérprete de Python normal, sin IDE ni notebook:

```
python analisis_desempeno.py
```

Tarda alrededor de cinco minutos. Genera en la carpeta `resultados_analisis/` las diez gráficas de este reporte y el archivo `resultados.json` con todos los indicadores. Todas las semillas aleatorias están fijas en 42, de modo que dos ejecuciones producen exactamente los mismos números. La separación de datos y el pipeline de preprocesamiento se importan de `main_framework.py`, por lo que el modelo analizado es exactamente el de la entrega anterior.